Generalized Additive Models

ECON 8310: Business Forecasting — Lecture 3

Department of Economics

University of Nebraska at Omaha

August 24, 2026

Lecture Outline

Motivation: Beyond Linear, Without Black Boxes

We want flexibility to capture nonlinear relationships while still being able to *explain* what the model is doing.

The Flexibility–Interpretability Tradeoff

Linear models (ARIMA, regression):

- ✓ Easy to interpret: each coefficient has a clear meaning
- ✓ Stable, well-understood properties
- × Assume a constant slope everywhere
- × Cannot capture nonlinear patterns
- × Miss threshold effects and curvature

Fully nonlinear models (trees, neural nets):

- ✓ Flexible — can capture almost any pattern
- ✓ High accuracy on complex data
- × Hard to interpret: why did the model make this prediction?
- × Interactions entangle all variables

GAMs offer a middle ground: nonlinear, flexible relationships for each variable, but the model stays **additive** — so each variable's effect can be visualized and explained independently.

Why Additivity Matters for Interpretation

In a non-additive model, the effect of one variable depends on the value of another — you cannot describe them separately. In a GAM, each variable's effect is isolated, making the model far easier to explain.

Optional math detail: Suppose we fit $y = x_1^2 x_2^2 + \varepsilon$. The marginal effect of x_1 is $\frac{\partial y}{\partial x_1} = 2x_1 x_2^2$, which depends on *both* x_1 and x_2 — you cannot describe the effect of x_1 without knowing x_2.

This depends on *both* x_1 and x_2 — you cannot describe the effect of x_1 without knowing x_2.

GAM Marginal Effects Are Simple

In a GAM: $y = f_1(x_1) + f_2(x_2) + \varepsilon$
The marginal effect of x_1 is just $f_1'(x_1)$ — a function of x_1 alone, regardless of x_2.

You can plot the effect of each variable separately and explain it to a business stakeholder.

GAM Structure and Smoothing

Additive nonlinear functions, penalized for wiggliness.

What Is a GAM?

Generalized Additive Model

$$y = \alpha + f_1(x_1) + f_2(x_2) + \cdots + f_k(x_k) + \varepsilon$$

where each $f_j(\cdot)$ is a **smooth, nonlinear function** estimated from the data (Wood 2017).

Key properties:

- Each $f_j(\cdot)$ is flexible — linear, curved, U-shaped, or threshold-like
- Additive — effects are *independent* and easy to visualize
- Each function minimizes prediction error subject to a smoothness penalty

Business Application

Modeling retail sales with:

- f_1(time): long-run trend
- f_2(month): seasonal effect
- f_3(unemployment): economic sensitivity

Each effect plotted and interpreted separately.

Controlling Smoothness: The Penalty Term

A GAM minimizes a **penalized** sum of squared errors:

$$\text{PSSE} = \underbrace{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}_{\text{fit}} + \underbrace{\lambda \int (f''(x))^2\, dx}_{\text{smoothness penalty}}$$

The penalty term:

- $f''(x)$ is the second derivative of f — measures how fast the slope is changing
- A wiggly function has a large integral; a smooth function has a small integral
- λ controls the trade-off: large λ = smoother; small λ = more flexible

$\lambda \to 0$: no penalty, fit as wiggly as needed — risk of overfitting

$\lambda \to \infty$: all f_j become linear — reduces to OLS

Optimal λ: chosen by **cross-validation** or GCV

Splines represent $f(x)$: piecewise polynomials that connect smoothly at **knot** points.

Prophet for Time Series GAMs

Facebook's Prophet decomposes a time series into trend, seasonality, and holiday effects using a GAM.

Prophet: Forecasting at Scale

Taylor and Letham (2018) developed Prophet for large-scale forecasting at Meta (Facebook). It models a time series as:

$$y(t) = \underbrace{g(t)}_{\text{trend}} + \underbrace{s(t)}_{\text{seasonality}} + \underbrace{h(t)}_{\text{holidays}} + \varepsilon_t$$

Key components:

- **Trend** $g(t)$**:** piecewise linear or logistic growth, with *changepoints* where the trend shifts
- **Seasonality** $s(t)$**:** Fourier series to capture weekly, monthly, yearly patterns
- **Holidays** $h(t)$**:** user-specified irregular events (Black Friday, COVID lockdowns)

Prophet is designed to be:

- Easy to use (one fit + forecast call)
- Robust to missing data and outliers
- Interpretable: each component plotted separately

`changepoint_prior_scale` controls trend flexibility: high = adapts quickly (risk of overfitting); low = stable trend.

Prophet: Code and Workflow

Prophet on Retail Sales

```
from prophet import Prophet
df = pd.DataFrame({'ds': dates, 'y': sales})
m = Prophet(
changepoint_prior_scale=0.05,
yearly_seasonality=True)
m.fit(df)
future = m.make_future_dataframe(
periods=12, freq='M')
forecast = m.predict(future)
m.plot_components(forecast)
```

Component plots:

- **Trend:** long-run direction with changepoints
- **Yearly seasonality:** seasonal wave across months
- **Weekly seasonality:** day-of-week pattern

When to use:

- ✓ Daily/weekly/monthly series with seasonality
- ✓ Known holiday effects; interpretable plots needed
- × Many correlated external predictors (use pyGAM)

pyGAM for General Additive Modeling

When you have multiple predictors and want flexible nonlinear effects for each.

pyGAM: Flexible Additive Modeling

pyGAM fits a GAM with user-specified smooth functions for each predictor:

pyGAM Example

```
from pygam import LinearGAM, s, f, l
# s()=spline  l()=linear  f()=factor
gam = LinearGAM(
s(0) + s(1) + s(2) + f(3))
gam = gam.gridsearch(X, y)
pdep, confi = gam.partial_dependence(
term=0, width=0.95)
y_hat = gam.predict(X_new)
```

Function types:

- `s(j)` Smooth (spline) function of feature j — flexible nonlinear effect
- `l(j)` Linear function of feature j
- `f(j)` Factor (binary / categorical) feature j

`gridsearch`: automatically searches for the optimal smoothness penalty λ using cross-validation.

Interpreting GAM Results: Partial Dependence Plots

Partial dependence plot for each predictor x_j:

- Shows the estimated function $\hat{f}_j(x_j)$ over the range of observed values
- Confidence bands show uncertainty around the smooth effect
- **Flat line:** x_j has no effect on y
- **Increasing curve:** x_j has a positive but potentially nonlinear effect
- **U-shaped:** x_j has an optimal level (too little or too much is bad)

World Happiness Example

pyGAM partial dependence for "freedom" predicting happiness:

- Near-linear effect: more freedom $\rightarrow$ more happiness
- Confidence band widens at extremes (sparse data)
- "Economy" effect: clearly nonlinear — diminishing returns at high income levels

Partial dependence plots make GAM results understandable to business stakeholders: *"As unemployment increases from 5% to 10%, predicted sales decline by X%."*

Key Takeaways and Roadmap

GAMs: flexible, interpretable, and underutilized in business forecasting.

GAMs vs. Other Methods: When to Choose Each

Method	Nonlinear	Interactions	Interpretable	Use case
ARIMA / ETS	No	No	High	Univariate TS
Linear regression	No	Manual	High	Few predictors
GAM	Yes	No	High	Few predictors, explain effects
Random Forest	Yes	Yes	Medium	Many predictors, tabular
XGBoost	Yes	Yes	Low	Best accuracy, tabular

Choose GAMs when:

- You need to **explain** the relationship between each predictor and the outcome
- Predictors have nonlinear, curved effects
- Time series has complex trend and seasonality patterns (use Prophet)
- Interpretability is more important than absolute prediction accuracy

Lecture 3: Key Takeaways

1. **GAMs** offer nonlinear flexibility while maintaining interpretability: $y = \alpha + f_1(x_1) + f_2(x_2) + \cdots + f_k(x_k) + \varepsilon$. Because the model is additive, each variable's effect can be visualized and explained independently.
2. **Smoothness penalty** (PSSE) prevents overfitting. The penalty λ controls the trade-off between fit and smoothness. It is chosen by cross-validation.
3. **Prophet** decomposes a time series into trend, seasonality, and holiday effects using a GAM. Ideal for business time series with seasonal patterns and known irregular events.
4. **pyGAM** fits a general GAM with any combination of predictors. Use `s()` for smooth effects, `l()` for linear, `f()` for categorical. `gridsearch` tunes λ automatically.
5. Partial dependence plots make results **actionable for stakeholders**: show clearly how each variable affects the forecast.

Next: Decision Trees — moving to tree-based methods that capture interactions automatically.

References I


Taylor, Sean J. and Benjamin Letham (2018). "Forecasting at Scale". In: *The American Statistician* 72.1, pp. 37–45.


Wood, Simon N. (2017). *Generalized Additive Models: An Introduction with R*. 2nd. Boca Raton, FL: Chapman and Hall/CRC.